ENTORNO DE DESARROLLO

1º DAW – (2025 – 2026)

IES Mar de Cádiz - El Puerto de Santa María

Ciclo de Desarrollo de aplicaciones web (DAW)

ESTELA PECCI PRIEGO

25 nov 2025

ESTELA PECCI PRIEGO

Tarea: Entregable Proyecto
Entorno de desarrollo

ÍNDICE

2. Introducción y Objetivos.....3
Objetivos..... 3 **3.**
Descripción del Diseño y Decisiones.....3
Decisiones de Diseño..... 3
Relaciones entre Clases..... 4 **4.**
Diagrama de Clases UML Completo..... **4**
Descripción de Clases y Visibilidad..... 5 **5.**
Descripción de la Estructura del Código..... **6 6.**
Conclusiones y Posibles Mejoras.....7
Conclusiones..... 7
Posibles Mejoras.....7

2. Introducción y Objetivos

Este proyecto tiene como fin implementar un sistema de gestión de pedidos en Java, utilizando los principios fundamentales de la Programación Orientada a Objetos (POO). El objetivo principal es modelar la realidad de un pequeño comercio electrónico donde existen diferentes tipos de productos (físicos y digitales) con distintas reglas de cálculo de precio final.

Objetivos

- **Demostrar el uso correcto de Herencia y Polimorfismo mediante una clase base abstracta (`Producto`).**
- **Implementar relaciones de Asociación y Agregación para modelar la relación entre un `Pedido`, un `Cliente` y múltiples `Productos`.**
- **Aplicar Encapsulamiento en todas las clases, protegiendo los atributos y exponiendo la funcionalidad mediante métodos getters y setters.**
- **Desarrollar una lógica de negocio clara para el cálculo de precios, diferenciando entre productos con coste de envío e IVA.**

3. Descripción del Diseño y Decisiones

El diseño del proyecto se centra en la modularidad y la extensibilidad, permitiendo añadir nuevos tipos de productos en el futuro sin modificar la estructura central.

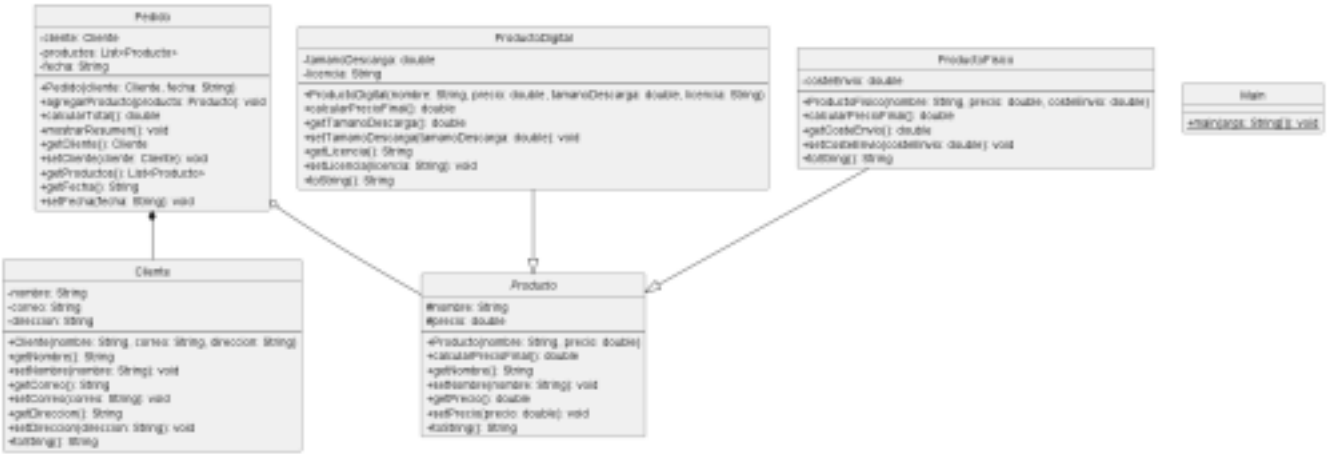
Decisiones de Diseño

1. **Clase Abstracta `Producto`: Se eligió una clase abstracta porque un "Producto" genérico no puede existir por sí mismo; debe ser específicamente físico o digital. Esto obliga a las subclases a implementar la lógica de precios.**
2. **Cálculo Polimórfico: El método `calcularPrecioFinal()` es abstracto en la clase base y se implementa de manera específica en las subclases:**
   - `ProductoFisico`: **Suma el `costeEnvio` al precio base.**
   - `ProductoDigital`: **Aplica un 21% de IVA al precio base (asumiendo impuestos estándar para servicios digitales).**
3. **Relación `Pedido-Producto` (Agregación): Un pedido contiene una lista de productos (`List<Producto>`). Se utiliza Agregación (la lista se inicializa dentro del constructor del `Pedido`) porque los productos pueden existir independientemente de un pedido concreto.**

Relaciones entre Clases

Relación	Clases Involucradas	Justificación
Herencia (Generalización)	Producto ProductoFisico, ProductoDigital	Permite el polimorfismo a través de calcularPrecioFinal() y reutiliza atributos comunes (nombre, precio).
Asociación	Pedido Cliente (1 a 1)	Un pedido se realiza *por* un cliente. El cliente existe independientemente del pedido.
Agregación Pedido	Producto (1 a N)	Un pedido está compuesto de cero o más productos. El Pedido mantiene y gestiona una colección de Productos.

4. Diagrama de Clases UML Completo



El siguiente diagrama visualiza la estructura estática del proyecto, incluyendo atributos, métodos y las relaciones clave (herencia, asociación, agregación).

Tarea: Entregable Proyecto
Entorno de desarrollo

Descripción de Clases y Visibilidad

Clase	Atributos (Visibilidad)	Métodos (Visibilidad)
Cliente	- nombre: String, - correo: String, - direccion: String	+ Cliente(String, String, String), + getNombre(), + setNombre(), + toString()
Producto (Abstracta)	# nombre: String, # precio: double	+ Producto(String, double), + calcularPrecioFinal(): double (Abstracto), + getNombre(), + setPrecio(), + toString()
ProductoFisico	- costeEnvio: double	+ ProductoFisico(String, double, double), + calcularPrecioFinal(): double (Implementa), + getCosteEnvio(), + toString()
ProductoDigital	- tamanoDescarga: double, - licencia: String	+ ProductoDigital(String, double, double, String), + calcularPrecioFinal(): double (Implementa), + getLicencia(), + toString()
Pedido	- cliente: Cliente, - productos: List<Producto>, - fecha: String	+ Pedido(Cliente, String), + agregarProducto(Producto), + calcularTotal(): double, + mostrarResumen(), + getCliente(), + getProductos()

5. Descripción de la Estructura del Código

Tarea: Entregable Proyecto
Entorno de desarrollo

Todo el código está contenido dentro del paquete `PROYECTO`**.**

- `Cliente.java`**: Clase simple con constructor y métodos getter/setter estándar. El método** `toString()` **proporciona una salida legible de los datos del cliente.** • `Producto.java`**: Define los campos protegidos (**`protected`**) para que sean accesibles directamente por las subclases, y declara el método abstracto** `calcularPrecioFinal()`**.**
- `ProductoFisico.java`**: Extiende** `Producto` **y utiliza** `super(nombre, precio)` **para inicializar la parte heredada. La implementación de** `calcularPrecioFinal()` **es sencilla:** `precio + costeEnvio`**.**
- `ProductoDigital.java`**: Extiende** `Producto`**. La implementación de** `calcularPrecioFinal()` **aplica el 21% de IVA:** `precio + (precio * 0.21)`**.** • `Pedido.java`**: Contiene la lógica de negocio central. Utiliza un** `ArrayList` **para manejar la colección de productos. El método** `calcularTotal()` **itera sobre la lista y llama polimórficamente a** `calcularPrecioFinal()` **en cada producto.** • `Main.java`**: Clase de demostración (driver). Instancia objetos, crea dos pedidos y muestra los resúmenes, verificando que el polimorfismo funciona correctamente al calcular los totales.**

6. Conclusiones y Posibles Mejoras

Conclusiones

El proyecto cumple satisfactoriamente con todos los objetivos de implementación de POO:

- **Se utilizó la herencia de forma efectiva para especializar el concepto de** `Producto`**.**
- **El polimorfismo permitió calcular los precios finales de manera correcta y flexible dentro de la clase** `Pedido`**.**
- **El diseño es sólido y extensible. Añadir un nuevo tipo de producto (ej.** `ProductoServicio` **con una tarifa fija) solo requeriría crear una nueva subclase sin tocar las clases existentes.**

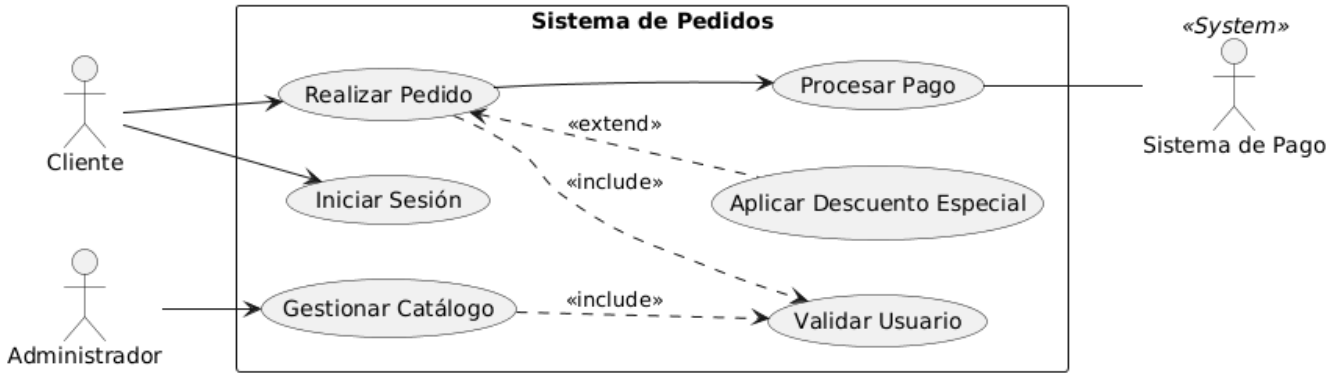
Tarea: Entregable Proyecto
Entorno de desarrollo
Posibles Mejoras

1. **Manejo de Cantidades: Actualmente, solo se pueden agregar unidades individuales de un producto. Se podría modificar el método** `agregarProducto` **en** `Pedido.java` **para aceptar una cantidad, o crear una clase intermedia** `LineaPedido` **(o** `DetallePedido`**) que contenga un** `Producto` **y una** `cantidad`**.**
2. **Tipos de IVA: El 21% de IVA es fijo. Se podría externalizar esta tasa a una constante, o permitir que la clase** `ProductoDigital` **maneje diferentes tipos de IVA (superreducido, reducido, general) según el país del cliente.**

3. **Fechas Reales: En lugar de usar** `String` **para la fecha, se debería utilizar la clase** `java.time.LocalDate` **para un manejo de fechas más robusto y preciso. 4. Persistencia de Datos: Los objetos se pierden al finalizar la ejecución. Una mejora fundamental sería implementar la persistencia de datos.**

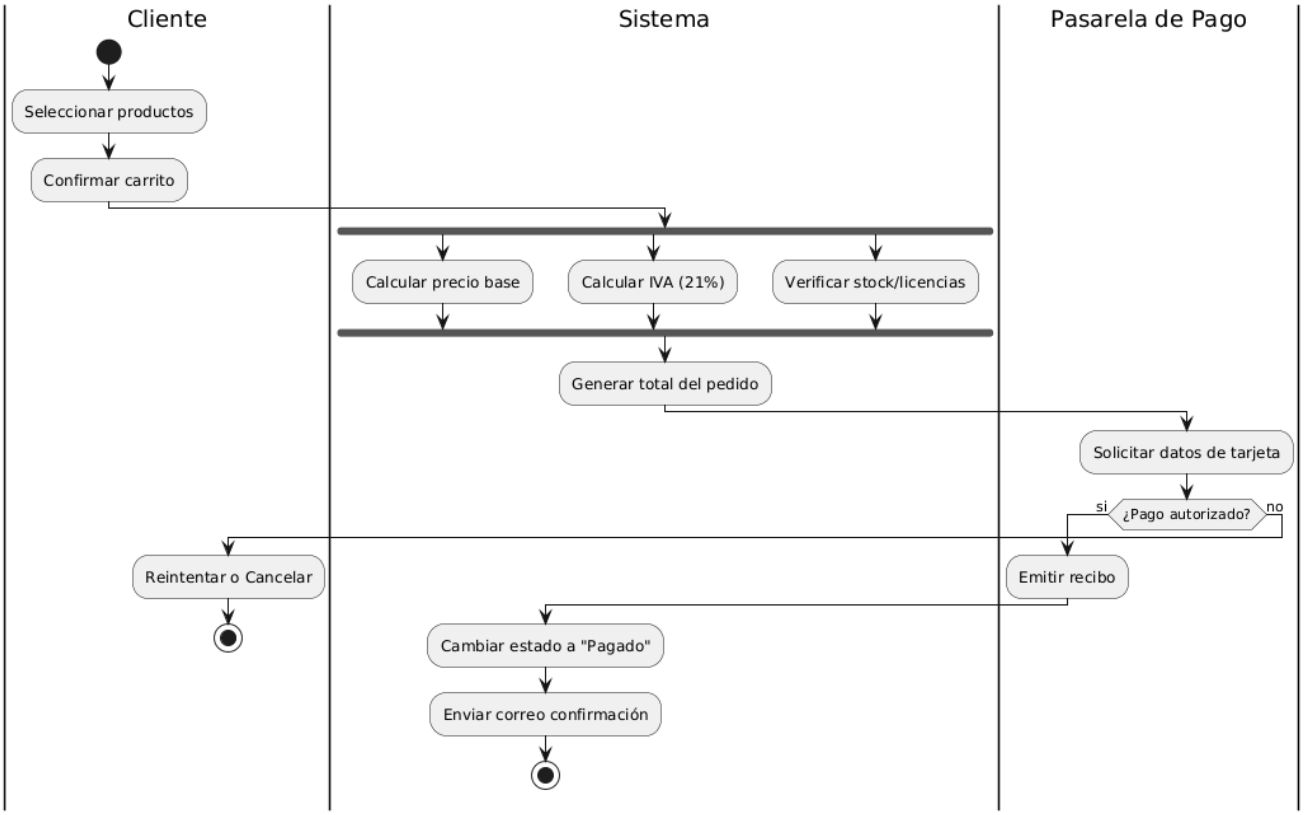
Parte 2:

a. Diagramas de Casos de Uso (Visión Externa)



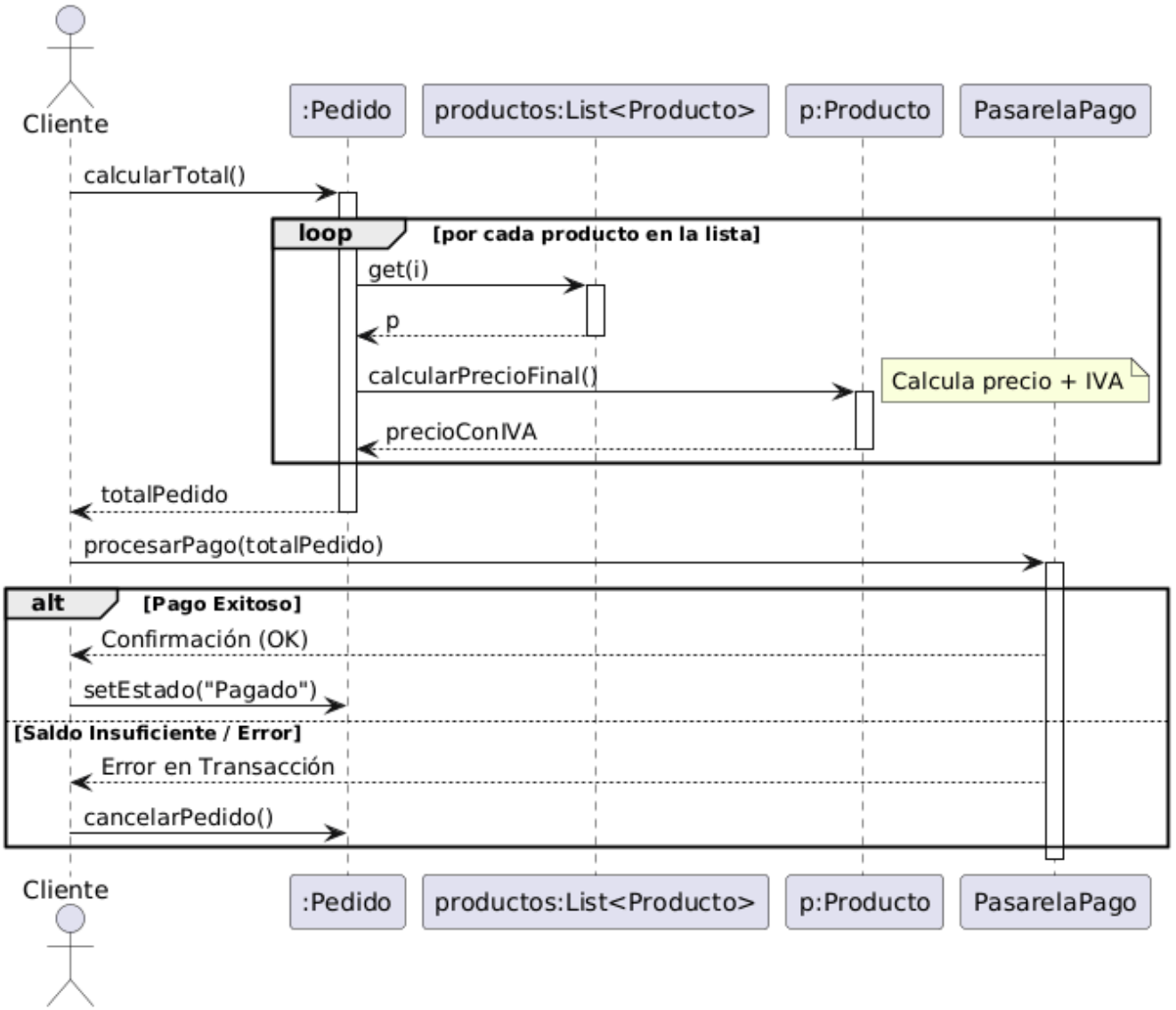
El diagrama muestra cómo un cliente, tras iniciar sesión, puede gestionar el catálogo, hacer un pedido y pasar por el pago, con la posibilidad de usar descuentos, mientras el administrador tiene un rol de supervisión en el sistema.

b. Diagramas de Actividades (Visión de Proceso)



Es un proceso paso a paso donde el cliente selecciona y confirma, el sistema prepara y calcula todo, y luego la pasarela de pago se encarga de recibir el pago y confirmar la compra.

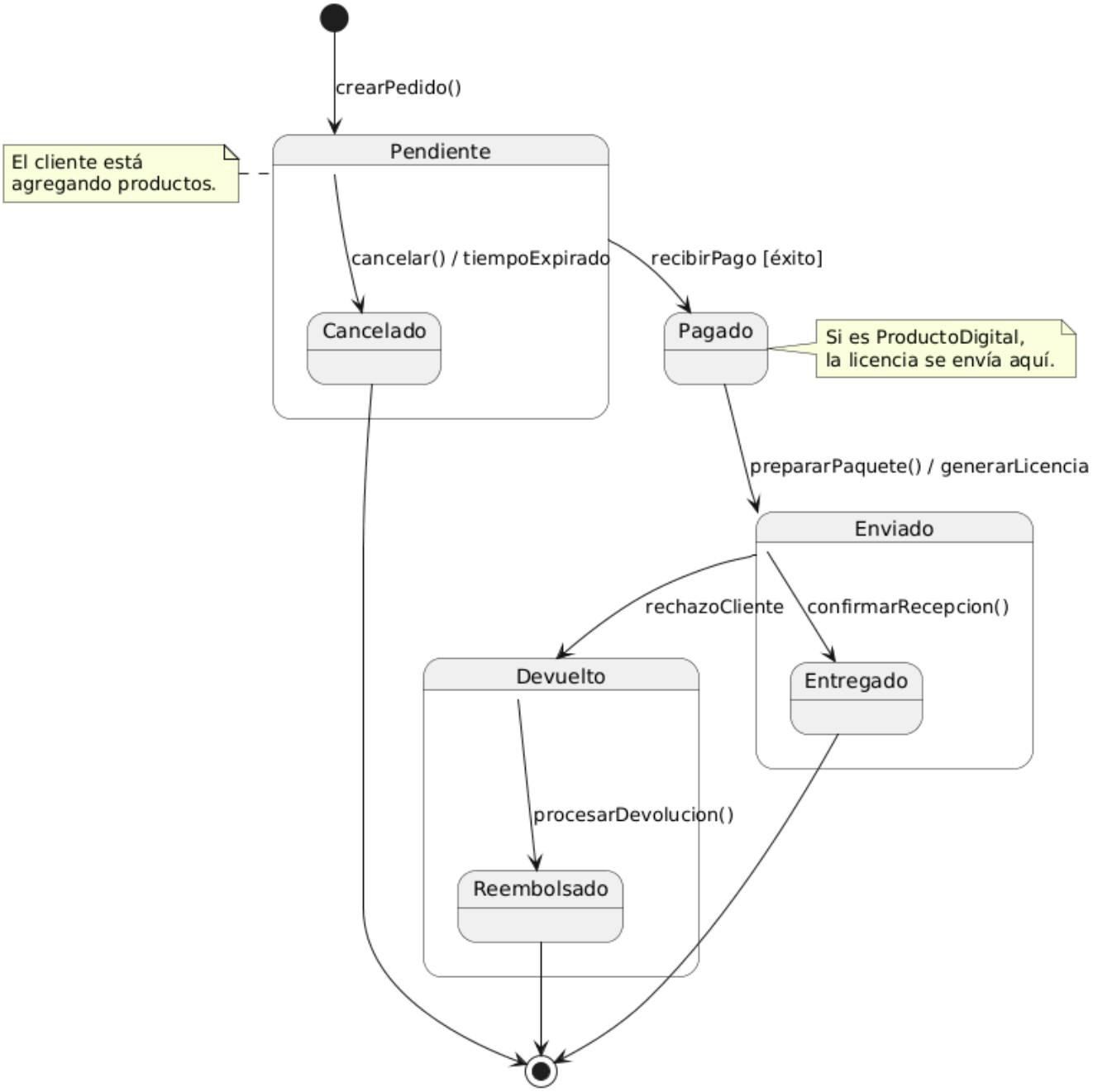
c. Diagramas de Secuencia (Visión de Interacción)



Este diagrama calcula el precio final del producto, suma todo para obtener el total a pagar, intenta cobrar ese total, dependiendo del resultado, confirma o cancela el pedido.

Todo esto pasa de forma automática desde que el cliente confirma hasta que recibe la confirmación o el error de pago.

d. Diagramas de Estados (Visión de Ciclo de Vida)



Un pedido puede terminar de tres formas distintas. Pendiente - Pagado - Enviado - Entregado.

Si se cancela antes de pagar, se queda en Cancelado. Y si el cliente lo devuelve, pasa por: Pagado - Enviado - Devuelto - Reembolsado.

Cada cambio depende de lo que haga el cliente o de lo que el sistema tenga que procesar.